

1

The bridge, from booked to what the feed posted

booked
5,800 over 5 orders

less never paid
800, order T-4

collected
5,000

plus posted twice
1,500, T-3 retry

posted in the feed
6,500

Invented numbers, on the two tiny tables. Booked comes from `orders` alone and posted from `payments` alone. The unpaid list is the anti-join, and the retry list is the same instalment posted twice.

Crux. Collected sits between booked and posted, and each step of the bridge is a list of rows someone can act on.

2

Which join answers which question

Join	The question it answers	Tiny rows
INNER	Which orders were matched, once per payment?	6
LEFT	What happened to every order, paid or not?	7
RIGHT	What happened to every payment, matched or not?	7
FULL OUTER	What is unmatched on either side?	8

Anand asked about every order, so his report is a LEFT JOIN with `orders` on the left.

Crux. Every join answers a question about the rows that do not match; choose the join by that question.

3

The grain rule, and the fan-out

Say what one row stands for before the join: one order in `orders`, one payment event in `payments`. An instalment order has two payment rows, so a join repeats its order and `SUM(o.amount)` counts it twice. Q2 turns 462 orders into 678 rows, and "collected" reads 1.96 times booked.

```
WITH paid_per_order AS (  
  SELECT order_id, SUM(amount) AS paid  
  FROM payments GROUP BY order_id  
  SELECT COUNT(*), SUM(o.amount),  
         SUM(COALESCE(pp.paid, 0))  
  FROM orders o  
  LEFT JOIN paid_per_order pp  
    ON pp.order_id = o.order_id  
  WHERE o.quarter = 'Q2';
```

Crux. A key that repeats on one side multiplies the other side's rows before any number is summed.

4

The reconciliation, written above the number

```
-- Rows in:  every Q2 order, from orders alone.  
-- Rows out: one row per order; equals rows in.  
-- Booked:   after the join equals from orders.  
-- The gap:  booked minus collected equals  
--           the booked value of the unpaid list.  
-- The feed: collected plus the retry surplus  
--           equals what payments posted.
```

If one line fails, the number stays in the team and the line that failed is named.

Crux. A join is done when its row count is explained: rows in, rows out, and the difference named.

5

WHERE against ON, and the anti-join

A payments condition in WHERE runs after the join and throws away the NULL rows of unpaid orders. In ON, it decides which payments attach, and every order survives.

```
LEFT JOIN payments p  
  ON p.order_id = o.order_id  
  AND p.paid_date BETWEEN '2026-07-01'  
                        AND '2026-09-30'
```

The anti-join is the one WHERE test on the right table that belongs there: `WHERE p.payment_id IS NULL`, or `WHERE NOT EXISTS (...)`, which reads as Anand's sentence.

Crux. A condition on the right-hand table belongs in the ON clause, or the LEFT JOIN becomes an INNER one.

6

A retry against an instalment

Pattern per order	What it is	What to do
Instalments 1 and 2	The customer paid as asked.	Count both payments.
Instalment 1 twice	The gateway retried.	Count it once and report the surplus.

`GROUP BY order_id HAVING COUNT(*) > 1` lists every instalment order as well, 216 in Q2. Group by `order_id`, `instalment_no` instead.

Crux. Two payment rows are not a double payment until the grain says they are the same payment.

THE WORDS ON THIS SHEET

- Grain** What one row of a table stands for, said before any join is written
- Fan-out** A join repeating a row once per match on a side whose key repeats
- INNER JOIN** A join that keeps only the rows that found a match on both sides
- LEFT JOIN** A join that keeps every left row, with NULLs where nothing matched
- Anti-join** A LEFT JOIN that keeps only the left rows whose right side is NULL
- Row-count reconciliation** Rows in, rows out and the difference named, written above the number
- Revenue bridge** Booked walked to posted in named moves, each move a list of rows
- Gateway retry** The same instalment posted twice by the payment feed